LAPORAN PRAKTIKUM

STRUKTUR DATA

“Abstract Data Type”

DISUSUN OLEH:

NOFRI ILHAM

2511531013

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T

Asisten Praktikum:

M Galid A

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Assalamu'alaikum Warahmatullahi Wabarakatuh

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum Struktur Data pertemuan pertama dengan materi *Abstract Data Type (ADT)* ini dapat diselesaikan dengan baik dan tepat waktu.

Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas praktikum sekaligus sebagai sarana untuk memperdalam pemahaman penulis mengenai konsep dasar *Abstract Data Type (ADT)* dalam bidang struktur data. Pada pertemuan ini, penulis mempelajari bagaimana suatu struktur data dapat direpresentasikan secara abstrak, yang menekankan pada operasi-operasi yang dapat dilakukan tanpa harus mengetahui detail implementasinya. Konsep ini sangat penting karena menjadi dasar dalam pengembangan program yang lebih modular, fleksibel, dan mudah untuk dikembangkan di masa mendatang.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat berbagai kekurangan, baik dari segi penulisan, penyusunan kalimat, maupun kelengkapan materi yang disajikan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari pembaca agar laporan ini dapat diperbaiki dan menjadi lebih baik di masa yang akan datang.

Akhir kata, penulis berharap laporan praktikum ini dapat memberikan manfaat, menambah wawasan, serta menjadi referensi dalam memahami konsep *Abstract Data Type (ADT)* bagi penulis sendiri maupun pembaca lainnya.

Padang, 31 maret 2026

Nofri Ilham

DAFTAR ISI

KATA PENGANTAR i
DAFTAR ISI ii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan..... 1
1.3 Manfaat 2
BAB II PEMBAHASAN 3
2.1 Pengertian Abstract Data Type 3
2.2 Implementasi ADT pada praktikum 3
2.2.1 Kode Pertama(Jam_2511531013) 3
2.2.2 Kode Kedua(JamDriver_2511531013)..... 6
2.2.3 Kode Ketiga(JamDriver2_2511531013)..... 7
BAB III PENUTUP..... 10
3.1 Kesimpulan..... 10
3.2 Saran..... 10

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, pengolahan data yang terstruktur dan efisien merupakan hal yang sangat penting. Seiring dengan semakin kompleksnya permasalahan yang harus diselesaikan menggunakan program komputer, dibutuhkan suatu konsep yang dapat membantu dalam mengelola data secara lebih sistematis. Salah satu konsep dasar yang berperan penting dalam hal ini adalah *Abstract Data Type (ADT)*.

Abstract Data Type (ADT) merupakan suatu konsep yang mendefinisikan tipe data berdasarkan operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memperhatikan bagaimana cara implementasinya secara detail. Dengan kata lain, ADT lebih menekankan pada "apa yang bisa dilakukan" oleh suatu data, bukan "bagaimana cara melakukannya". Konsep ini sangat membantu programmer dalam merancang program karena dapat memisahkan antara logika penggunaan data dan implementasi teknisnya.

Pemahaman mengenai ADT menjadi sangat penting bagi mahasiswa informatika, terutama sebagai dasar dalam mempelajari berbagai struktur data seperti stack, queue, linked list, dan lainnya. Tanpa memahami konsep ADT, mahasiswa akan kesulitan dalam memahami bagaimana suatu struktur data bekerja secara konseptual dan bagaimana menggunakannya secara efektif dalam pemrograman.

Pada praktikum pertemuan pertama ini, mahasiswa diperkenalkan dengan konsep dasar ADT agar memiliki fondasi yang kuat dalam memahami materi struktur data selanjutnya. Melalui praktikum ini, diharapkan mahasiswa tidak hanya memahami teori, tetapi juga mampu mengaplikasikan konsep ADT dalam bentuk latihan sederhana, sehingga dapat meningkatkan kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

1.2 Tujuan

Adapun tujuan dari praktikum Struktur Data pertemuan pertama dengan materi *Abstract Data Type (ADT)* adalah sebagai berikut:

1. Memahami konsep dasar *Abstract Data Type (ADT)* dalam struktur data.
2. Mengetahui perbedaan antara konsep abstrak dan implementasi dalam pemrograman.

3. Mampu mengidentifikasi operasi-operasi yang dapat dilakukan pada suatu tipe data.
4. Melatih kemampuan berpikir logis dan sistematis dalam memahami suatu permasalahan.
5. Mampu mengaplikasikan konsep ADT dalam bentuk latihan atau studi kasus sederhana.

1.3 Manfaat

Adapun manfaat yang diperoleh dari praktikum ini adalah sebagai berikut:

1. Mahasiswa dapat memahami dasar-dasar struktur data dengan lebih baik, khususnya konsep *Abstract Data Type (ADT)*.
2. Membantu mahasiswa dalam merancang program yang lebih terstruktur dan modular.
3. Meningkatkan kemampuan analisis dalam menyelesaikan masalah yang berkaitan dengan pengolahan data.
4. Menjadi dasar untuk mempelajari struktur data lanjutan seperti stack, queue, dan linked list.
5. Meningkatkan keterampilan praktis mahasiswa dalam mengimplementasikan konsep teori ke dalam pemrograman.

BAB II
PEMBAHASAN

2.1 Pengertian Abstract Data Type

Tipe data abstrak (ADT) dapat didefinisikan sebagai *model matematika dari objek data yang menyempurnakan tipe data dengan cara mengaitkannya dengan fungsi-fungsi yang beroprasi pada data yang bersangkutan*. Merupakan hal yang sangat penting untuk mengenali bahwa operasi-operasi yang akan dimanipulasi data pada objek yang bersangkutan termuat dalam spesifikasi ADT. Sebagai contoh, ADT HIMPUNAN didefinisikan sebagai koleksi data yang diakses oleh operasi-operasi himpunan seperti penggabungan (UNION), irisan (INTERSECTION), dan selisih antar-himpunan (SET DIFFERENCE).

Implementasi dari ADT harus menyediakan cara tertentu untuk merepresentasikan unsur tipe data (seperti matrix) dan cara untuk mengimplementasikan operasi -operasi matrix. Secara tipikal, kita akan mendeskripsikan operasi-operasi pada ADT dengan **algoritma** (logika berfikir) tertentu. Algoritma ini biasanya berupa urutan instruksi yang menspesifikasi secara tepat bagaimana operasi-operasi akan dilakukan/dieksekusi oleh komputer.

2.2 Implementasi ADT pada praktikum

Pada pekan pertama praktikum Struktur Data,mahasiswa diinstruksikan untuk menulis ulang kode yang telah tertera di papan tulis . Berikut merupakan kode tersebut

2.2.1 Kode Pertama(Jam_2511531013)

```java
package pekan1_2511531013;

public final class Jam_2511531013 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    //-- Validator ---
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) &&
               (0 <= m && m <= 59) &&
               (0 <= s && s <= 59);
    }

    // --- Konstruktor ---
    public Jam_2511531013(int h, int m, int s) {
        this.hh = h;
        this.mm = m;
        this.ss = s;
    }

    // --- Selector ---
    public int getHour() { return hh; }
    public int getMinute() { return mm; }
    public int getSecond() { return ss; }

    // --- Mutator ---
    public void setHour(int h) { this.hh = h; }
    public void setMinute(int m) { this.mm = m; }
    public void setSecond(int s) { this.ss = s; }

    // --- Konversi ---
    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss;
    }
```

```java
    public static Jam_2511531013 fromSeconds(int total) {
        if (total < 0) throw new IllegalArgumentException("detik negatif");

        total %= 24 * 3600;
        int h = total / 3600;
        total %= 3600;
        int m = total / 60;
        int s = total % 60;

        return new Jam_2511531013(h, m, s);
    }

    // --- Relasional ---
    public int compareTo(Jam_2511531013 other) {
        return Integer.compare(this.toSeconds(), other.toSeconds());
    }


    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511531013 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
    }


    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss);
    }

    // --- Aritmatika ---
    public Jam_2511531013 plus(Jam_2511531013 other) {
        return fromSeconds(this.toSeconds() + other.toSeconds());
    }

    public Jam_2511531013 minus(Jam_2511531013 other) {
        return fromSeconds(Math.floorMod(
            this.toSeconds() - other.toSeconds(),
```

```java
            24 * 3600
        ));
    }

    public Jam_2511531013 nextSecond() {
        return fromSeconds(this.toSeconds() + 1);
    }

    public Jam_2511531013 nextSeconds(int n) {
        return fromSeconds(this.toSeconds() + Math.max(0, n));
    }

    public Jam_2511531013 prevSecond() {
        return fromSeconds(Math.floorMod(
            this.toSeconds() - 1,
            24 * 3600
        ));
    }

    public Jam_2511531013 prevnSeconds(int n) {
        return fromSeconds(Math.floorMod(
            this.toSeconds() - Math.max(0, n),
            24 * 3600
        ));
    }

    // --- Durasi ---
    public static int durasiDetik(Jam_2511531013 awal, Jam_2511531013 akhir) {
        return akhir.toSeconds() - awal.toSeconds();
    }

    // --- Output ---

    public String toString() {
        return String.format("%02d:%02d:%02d", hh, mm, ss);
    }
}
```

Program diatas merupakan implementasi *Abstract Data Type (ADT)* untuk merepresentasikan waktu yang terdiri dari jam, menit, dan detik. Kelas ini memiliki atribut hh, mm, dan ss yang bersifat private sebagai bentuk *encapsulation*, sehingga data tidak dapat diakses langsung dari luar kelas.

Program ini menyediakan berbagai method seperti *constructor* untuk inisialisasi objek, *getter* dan *setter* untuk mengakses serta mengubah nilai, serta method isValid untuk memvalidasi waktu. Selain itu, terdapat fungsi konversi seperti toSeconds untuk mengubah waktu menjadi detik dan fromSeconds untuk mengubah detik kembali menjadi format jam.

Kelas ini juga mendukung operasi perbandingan melalui method compareTo dan equals, serta operasi aritmatika waktu seperti penjumlahan (plus), pengurangan (minus), penambahan detik (nextSecond, nextSeconds), dan pengurangan detik

(prevSecond, prevnSeconds). Terdapat juga method durasiDetik untuk menghitung selisih waktu dalam detik.

Dengan adanya class ini, pengolahan data waktu menjadi lebih mudah, terstruktur, dan sesuai dengan konsep ADT karena pengguna hanya perlu menggunakan operasi yang tersedia tanpa mengetahui detail implementasinya.

2.2.2 Kode Kedua(JamDriver_2511531013)

```java
package pekan1_2511531013;

public class JamDriver_2511531013 {
    public static void main(String[] args) {
        Jam_2511531013 a=new Jam_2511531013(23,59,50);
        Jam_2511531013 b=new Jam_2511531013(0,0,15);
        System.out.println("a              ="+a);
        System.out.println("b              ="+b);
        System.out.println("a+b            ="+a.plus(b));
        System.out.println("next 20s       ="+a.nextSeconds(20));
        System.out.println("durasi(a,b)    ="+ Jam_2511531013.durasiDetik(a,b));
        System.out.println("a.compareTo(b)="+a.compareTo(b));

    }

}
```

Kode JamDriver2_2511531013 berfungsi sebagai program utama untuk menjalankan dan menguji class Jam_2511531013 yang telah dibuat sebelumnya. Program ini menerima input dari pengguna berupa jam, menit, dan detik untuk membentuk dua objek waktu, yaitu Jam 1 dan Jam 2, dengan bantuan Scanner.

Sebelum objek dibuat, program akan mengecek terlebih dahulu apakah input yang dimasukkan sudah sesuai menggunakan method isValid. Jika tidak valid, maka pengguna diminta untuk mengulang input. Setelah data benar, program akan menampilkan kedua waktu dalam format jam serta dalam bentuk total detik melalui method toSeconds.

Selanjutnya, program membandingkan kedua waktu menggunakan method compareTo untuk mengetahui urutan waktunya. Program juga menghitung selisih waktu dalam satuan detik menggunakan method durasiDetik. Selain itu, terdapat beberapa operasi lain seperti menampilkan waktu setelah ditambah satu detik (nextSecond), waktu sebelum satu detik (prevSecond), serta hasil penjumlahan dua waktu (plus).

2.2.3 Kode Ketiga(JamDriver2_2511531013)

```java
package pekan1_2511531013;

import java.util.Scanner;

public class JamDriver2_2511531013 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in);
        System.out.println("=== Program Driver Objek Jam ===");

        // 1. Input jam pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511531013 j1 = buatJamDariInput(input);

        // 2. Input jam kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511531013 j2 = buatJamDariInput(input);

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " + j1);
        System.out.println("Jam 2 (String)      : " + j2);
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
        System.out.println("Jam 2 dalam detik   : " + j2.toSeconds());

        // 4. Perbandingan
        int perbandingan = j1.compareTo(j2);
        if (perbandingan > 0) {
            System.out.println("Status : Jam 1 lebih lambat (setelah) Jam 2");
        } else if (perbandingan < 0) {
            System.out.println("Status : Jam 1 lebih awal (sebelum) Jam 2");
        } else {
            System.out.println("Status : Jam 1 dan Jam 2 sama");
        }

        // 5. Operasi Aritmatika
```

```java
        System.out.println("Durasi (J1 ke J2) : " +
                Jam_2511531013.durasiDetik(j1, j2) + " detik");

        Jam_2511531013 jNext = j1.nextSecond();
        System.out.println("Detik berikutnya : " + jNext);

        Jam_2511531013 jPrev = j1.prevSecond();
        System.out.println("Detik sebelumnya : " + jPrev);

        // 6. Penjumlahan
        Jam_2511531013 jHasil = j1.plus(j2);
        System.out.println("J1 + J2 : " + jHasil);

        input.close();
        System.out.println("\nProgram Selesai.");
    }

    private static Jam_2511531013 buatJamDariInput(Scanner sc) {
        int h, m, s;

        while (true) {
            System.out.print("Masukkan jam (0-23): ");
            h = sc.nextInt();

            System.out.print("Masukkan menit (0-59): ");
            m = sc.nextInt();

            System.out.print("Masukkan detik (0-59): ");
            s = sc.nextInt();

            if (Jam_2511531013.isValid(h, m, s)) {
                return new Jam_2511531013(h, m, s);
            } else {
                System.out.println("[Error] Input tidak valid! Ulangi.\n");
            }
        }
    }
```

Kode diatas digunakan sebagai penghubung (driver) untuk menjalankan dan menguji fungsi-fungsi yang terdapat pada class Jam_2511531013. Program ini meminta pengguna untuk memasukkan dua data waktu yang terdiri dari jam, menit, dan detik melalui input keyboard.

Setiap input yang diberikan akan diperiksa terlebih dahulu menggunakan method isValid agar sesuai dengan format waktu yang benar. Jika input tidak valid, maka pengguna diminta untuk mengulang hingga data yang dimasukkan benar. Setelah itu, program akan menampilkan kedua waktu tersebut dalam bentuk format jam serta dalam bentuk total detik.

Program juga melakukan perbandingan antara dua waktu menggunakan method compareTo untuk mengetahui apakah waktu pertama lebih awal, lebih lambat, atau sama dengan waktu kedua. Selain itu, dihitung juga selisih waktu dalam satuan detik menggunakan method durasiDetik.

Tidak hanya itu, program menampilkan hasil beberapa operasi lain seperti waktu setelah ditambah satu detik (nextSecond), waktu sebelum satu detik (prevSecond), serta hasil penjumlahan dua waktu menggunakan method plus.

BAB III
PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum Struktur Data pekan pertama dengan materi *Abstract Data Type (ADT)*, dapat disimpulkan bahwa ADT merupakan konsep dasar yang sangat penting dalam pemrograman, khususnya dalam pengelolaan data secara terstruktur. ADT menekankan pada operasi yang dapat dilakukan terhadap suatu data tanpa harus mengetahui detail implementasinya, sehingga memudahkan dalam pengembangan program yang lebih modular dan fleksibel.

Melalui praktikum ini, mahasiswa tidak hanya memahami konsep ADT secara teori, tetapi juga mampu mengimplementasikannya dalam bentuk program sederhana, seperti pembuatan class **Jam** beserta berbagai operasinya. Implementasi tersebut menunjukkan bagaimana suatu tipe data dapat dikemas secara abstrak dengan berbagai method seperti validasi, konversi, perbandingan, serta operasi aritmatika waktu.

Selain itu, penggunaan program driver membantu dalam menguji dan memahami cara kerja ADT secara lebih nyata melalui interaksi langsung dengan input pengguna. Dengan demikian, praktikum ini berhasil meningkatkan pemahaman mahasiswa terhadap konsep dasar struktur data serta melatih kemampuan berpikir logis, sistematis, dan terstruktur dalam menyelesaikan permasalahan.

3.2 Saran

Adapun saran yang dapat diberikan setelah melaksanakan praktikum ini adalah sebagai berikut:

1. Mahasiswa diharapkan lebih banyak berlatih dalam mengimplementasikan konsep ADT dengan berbagai studi kasus yang berbeda agar pemahaman semakin mendalam.
2. Dalam penulisan program, sebaiknya lebih memperhatikan kerapihan kode serta penggunaan nama variabel dan method yang jelas agar mudah dipahami.
3. Asisten praktikum dapat memberikan lebih banyak contoh kasus atau latihan tambahan agar mahasiswa semakin terbiasa dengan penerapan konsep ADT.

4. Praktikum selanjutnya diharapkan dapat lebih menantang dengan mengaitkan ADT ke struktur data yang lebih kompleks seperti stack, queue, dan linked list.
5. Mahasiswa juga disarankan untuk mempelajari kembali materi sebelum praktikum agar proses pembelajaran saat praktikum menjadi lebih efektif.

DAFTAR PUSTAKA

[1] Shoffin, “Perulangan While dan Do While,” [Daring]. Tersedia pada: https://shoffin.wordpress.com/2010/08/25/14/ [Diakses: 8-Nov-2025].